

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования “Санкт-Петербургский политехнический университет
Петра Великого”

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчет по курсовой работе

Разработка игрового бота для расширенной версии игры “Крестики-нолики”
по дисциплине “Структуры данных”

Выполнил:

Студент группы 5130201/50302

_____ Кузнецов М.С.

Проверил:

Преподаватель

_____ Сеннов Н.В.

“ ____ ” _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Математическое описание алгоритма	5
2.1 Модель поля и допустимого хода	5
2.2 Правило победы для игроков X и O	5
2.3 Свободные области и центр кластера	5
2.4 Множество ходов-кандидатов	5
2.5 Первый ход	6
2.6 Оценка отрезков длины 5	6
2.7 Тактические угрозы и итоговая оценка хода	6
2.8 Итоговый алгоритм выбора хода	7
2.9 Обоснование выбранного подхода	7
3 Особенности реализации	8
4 Результаты работы программы	9
Заключение	10
Список литературы	11
A Исходный код заголовочного файла my_player.hpp	12
B Исходный код файла реализации my_player.cpp	13

Введение

В данной работе рассматривается разработка игрового бота для расширенной версии игры “Крестики-нолики”. В отличие от классической игры на поле 3×3 , рассматриваемая версия проводится на поле 20×20 , может содержать препятствия, а победной считается непрерывная линия из пяти одинаковых символов по горизонтали, вертикали или диагонали. Из-за большого размера поля простой полный перебор продолжений быстро становится непрактичным, поэтому при проектировании игрока использовались эвристические методы, основанные на оценке локальных и позиционных признаков. Такой подход соответствует общим принципам построения алгоритмов для дискретных задач и методам принятия решений в игровых задачах искусственного интеллекта [1, 2].

Предметная область работы относится к классу комбинаторных игр на клеточном поле. В таких играх качество решения определяется не только формальной проверкой победной линии, но и способностью алгоритма заранее оценивать угрозы, перспективные направления развития атаки и влияние занятых клеток на дальнейшие ходы [3]. Для рассматриваемой игры дополнительно важно асимметричное правило завершения партии: игрок *O* выигрывает сразу после построения линии, а игрок *X* после своей линии должен учитывать последний ответный ход игрока *O*. Эта особенность меняет оценку позиции и делает стратегию для двух знаков различной.

Цель работы состоит в разработке бота MyPlayer, который по текущему состоянию поля выбирает допустимую клетку для хода за ограниченное время. Для достижения этой цели в работе формализована модель поля, описано множество рассматриваемых ходов, введены оценки атаки и защиты, реализована обработка немедленных побед и блокировок, а также проведены серии автоматических игр против базовых игроков. Исходный код был написан на C++, а при описании реализации использовались справочные материалы по языку [4]. При редактировании пояснительного текста применялся инструмент ChatGPT¹.

¹OpenAI. ChatGPT: программный инструмент на основе большой языковой модели [Электронный ресурс]. URL: <https://openai.com/chatgpt/> (дата обращения: 10.05.2026).

1. Постановка задачи

Необходимо реализовать игрока для уже заданной игровой среды расширенных “Крестиков-ноликов”. Логика игрового движка, смена ходов, хранение истории партии и определение общего результата игры не разрабатываются заново; бот получает состояние партии через предоставленный интерфейс и должен вернуть только один следующий ход. Таким образом, задача состоит не в реализации всей игры, а в разработке алгоритма выбора клетки для текущего игрока.

На вход алгоритму подаются размеры поля N и M , матрица состояния клеток A , знак текущего игрока $s \in \{X, O\}$, а также информация о клетках, недоступных для хода. В данной работе эксперименты проводятся для поля 20×20 . Каждая клетка может быть свободной, занятой символом X , занятой символом O или заблокированной препятствием. Ход допустим только в свободную клетку внутри границ поля. На выходе алгоритм должен выдать координаты одной допустимой клетки (i^*, j^*) , в которую игровой движок поставит символ текущего игрока.

Алгоритм должен учитывать пять групп условий. Во-первых, он обязан возвращать только допустимый ход. Во-вторых, при наличии немедленной победы текущего игрока ход должен выбираться с учётом правила последнего ответа для X . В-третьих, при отсутствии собственной немедленной победы необходимо блокировать ход соперника, который приводит к его победной линии. В-четвёртых, обычные ходы должны сравниваться по численной оценке, включающей атакующие возможности, защиту, близость к активной области и влияние препятствий. В-пятых, алгоритм должен работать достаточно быстро для серийного запуска партий, поэтому перебор всех игровых продолжений не используется.

К сдаче предъявляются количественные требования по скорости и силе игры. Алгоритм должен быть эффективным: время вычисления одного хода должно составлять величину порядка 50 — 100 мс на рабочей станции или ноутбуке. Алгоритм также должен быть достаточно сильным: процент побед над игроком базового уровня сложности без учёта ничьих должен быть больше 50%, а над игроком повышенного уровня сложности — больше 25%. Оба базовых игрока предоставляются как подключаемая библиотека с закрытым исходным кодом, которая подключается к проекту; соответствующий

релиз библиотеки доступен по адресу: <https://github.com/vsennov/tictactoe-course/releases/tag/v1.1.0>.

Результатом работы является программная реализация бота MyPlayer и экспериментальная проверка его поведения. Корректность решения оценивается по допустимости возвращаемых ходов, выполнению указанных требований к скорости и силе игры, а также по среднему времени выбора хода.

2. Математическое описание алгоритма

2.1. Модель поля и допустимого хода

Игровое поле рассматривается как матрица

$$A = (a_{ij}), \quad i = 1, 2, \dots, N, \quad j = 1, 2, \dots, M, \quad (1)$$

где a_{ij} обозначает состояние клетки в строке i и столбце j . Множество возможных состояний клетки задаётся как

$$S = \{\emptyset, X, O, B\}, \quad (2)$$

где $\emptyset$ означает свободную клетку, X и O — символы игроков, а B — препятствие. Ход (i, j) допустим тогда и только тогда, когда $1 \leq i \leq N$, $1 \leq j \leq M$ и $a_{ij} = \emptyset$.

Для проверки линий используются четыре направления

$$D = \{(1, 0), (0, 1), (1, 1), (1, -1)\}. \quad (3)$$

Если для некоторого направления $(d_i, d_j) \in D$ пять последовательных клеток

$$(i, j), (i + d_i, j + d_j), (i + 2d_i, j + 2d_j), (i + 3d_i, j + 3d_j), (i + 4d_i, j + 4d_j)$$

находятся внутри поля и содержат один и тот же знак игрока, то этот игрок построил победную линию длины 5.

2.2. Правило победы для игроков X и O

Пусть $L_s(A)$ означает наличие на поле A линии из пяти символов игрока s . Для игрока O победа определяется прямым условием

$$\text{Win}_O(A) \iff L_O(A). \quad (4)$$

Для игрока X учитывается последний ответ игрока O . Обозначим через $R_O(A)$ количество свободных клеток, ход в которые позволяет игроку O немедленно построить линию из пяти символов. Тогда гарантированная победа игрока X

задаётся условием

$$\text{Win}_X(A) \iff L_X(A) \wedge R_O(A) = 0. \quad (5)$$

Если после хода X линия X уже построена, но $R_O(A) > 0$, такой ход рассматривается не как гарантированная победа, а как позиция, в которой O может последним ответом добиться ничьей. Поэтому в оценочной функции для X защитная составляющая имеет больший вес, чем для O .

2.3. Свободные области и центр кластера

Из-за препятствий геометрический центр поля не всегда является хорошей областью для первого хода и для позиционного выбора. Поэтому вводится понятие свободного кластера. Свободный кластер — это максимальное множество свободных клеток, в котором любые две клетки связаны цепочкой соседних свободных клеток. Соседними считаются клетки, имеющие общую сторону или вершину, то есть используется связность по восьми направлениям.

Пусть $K = \{(i_1, j_1), \dots, (i_k, j_k)\}$ — крупнейший свободный кластер. Его средняя точка определяется формулами

$$\bar{i} = \frac{1}{k} \sum_{t=1}^k i_t, \quad \bar{j} = \frac{1}{k} \sum_{t=1}^k j_t. \quad (6)$$

Центром кластера считается такая свободная клетка $(i_c, j_c) \in K$, для которой евклидово расстояние до точки $(\bar{i}, \bar{j})$ минимально. Если несколько клеток имеют одинаковое расстояние, выбирается клетка с большим количеством потенциальных отрезков длины 5 и меньшим штрафом за близость к препятствиям.

2.4. Множество ходов-кандидатов

Пусть F — множество всех свободных клеток поля:

$$F = \{(i, j) \mid a_{ij} = \emptyset\}.$$

После начала партии алгоритм не оценивает все клетки из F , потому что удалённые от уже поставленных символов клетки редко влияют на ближайшую тактику. Множество кандидатов C определяется как

$$C = \{(i, j) \in F \mid \exists(p, q) : a_{pq} \in \{X, O\}, |i - p| \leq 2, |j - q| \leq 2\}. \quad (7)$$

Если на поле ещё нет символов игроков, то используется специальное правило первого хода. Если множество C оказалось пустым из-за изолированного расположения свободных клеток, алгоритм возвращается к рассмотрению всех клеток из F .

2.5. Первый ход

Для первого хода нельзя использовать близость к уже поставленным символам, поэтому каждая свободная клетка оценивается по трём параметрам. Пусть $L(i, j)$ — количество отрезков длины 5, которые проходят через клетку (i, j) и полностью состоят из свободных клеток; $P(i, j)$ — штраф за препятствия в окрестности радиуса 2; $Q(i, j)$ — манхэттенское расстояние до центра крупнейшего свободного кластера. Тогда оценка первого хода равна

$$F_0(i, j) = 1000L(i, j) - 20P(i, j) - Q(i, j). \quad (8)$$

Первым ходом выбирается клетка

$$(i^*, j^*) = \arg \max_{(i, j) \in F} F_0(i, j). \quad (9)$$

Такая формула задаёт все используемые веса явно: потенциальная линия получает вес 1000, близость к препятствию штрафует с коэффициентом 20, а расстояние до центра кластера используется как слабый позиционный критерий.

2.6. Оценка отрезков длины 5

Для оценки атаки рассматриваются все отрезки длины 5 в направлениях из формулы (3). Для выбранного игрока s в каждом отрезке подсчитываются три величины: k — количество символов игрока s , e — количество свободных

клеток, b — количество заблокированных клеток. К заблокированным относятся символы соперника и препятствия. Оценка одного отрезка равна

$$\text{Score}(k, e, b) = \begin{cases} 0, & b > 0, \\ 100000000, & k \geq 5, \\ 3000000, & k = 4, e = 1, \\ 80000, & k = 3, e = 2, \\ 3000, & k = 2, e = 3, \\ 30, & k = 1, e = 4, \\ 0, & \text{иначе.} \end{cases}$$

Атакующая ценность позиции для игрока s определяется как сумма оценок по всем подходящим отрезкам:

$$A(A, s) = \sum \text{Score}(k, e, b).$$

2.7. Тактические угрозы и итоговая оценка хода

Пусть A' — состояние поля после предполагаемого хода текущего игрока s в клетку (i, j) , а $\bar{s}$ — знак соперника. Через $R_s(A')$ обозначается количество ходов, после которых игрок s может немедленно построить линию из пяти символов. Тактическая часть оценки задаётся формулами

$$T_{\text{danger}}(i, j) = -500000000 \cdot R_{\bar{s}}(A'), \quad (10)$$

$$T_{\text{attack}}(i, j) = \begin{cases} 300000000, & R_s(A') \geq 2, \\ 3000000, & R_s(A') = 1, \\ 0, & R_s(A') = 0, \end{cases}$$

$$T(i, j) = T_{\text{attack}}(i, j) + T_{\text{danger}}(i, j).$$

Позиционная близость к активной зоне учитывается функцией $N(i, j)$. В окрестности радиуса 2 собственный символ на манхэттенском расстоянии 1 даёт 30 баллов, собственный символ на расстоянии 2 даёт 8 баллов, символ соперника на расстоянии 1 даёт 15 баллов, а символ соперника на расстоянии

2 даёт 4 балла. Близость к центру свободного кластера задаётся функцией

$$Q_c(i, j) = -(|i - i_c| + |j - j_c|).$$

Итоговая оценка различается для игроков X и O :

$$F_O(i, j) = T(i, j) + 22A(A', O) - 24A(A', X) + N(i, j) + Q_c(i, j), \quad (11)$$

$$F_X(i, j) = T(i, j) + 12A(A', X) - 24A(A', O) + N(i, j) + Q_c(i, j). \quad (12)$$

Игрок O получает более высокий атакующий коэффициент, поскольку его линия немедленно завершает партию. Игрок X сильнее ориентирован на защиту, так как после его линии возможен последний ответ O .

2.8. Итоговый алгоритм выбора хода

На вход алгоритму подаются матрица A , размеры поля N и M , знак текущего игрока s и множество состояний клеток из формулы (2). На выходе алгоритм выдаёт координаты допустимого хода (i^*, j^*) . Порядок выбора хода следующий.

1. Найти крупнейший свободный кластер и его центр по формуле (6).
2. Если на поле нет символов X и O , выбрать первый ход по формулам (8) и (9) и завершить алгоритм.
3. Сформировать множество ходов-кандидатов C по формуле (7).
4. Для каждого хода из C проверить, создаёт ли он немедленную победу текущего игрока. Для O используется условие (4); для X используется условие (5). Если такой ход существует, выбрать его и завершить алгоритм.
5. Если собственной немедленной победы нет, проверить, существует ли ход соперника, который на следующем ходе создаёт его победную линию. Если такая клетка найдена, занять её текущим ходом.

6. Если срочного хода нет, для каждой клетки из C построить временное состояние A' и вычислить оценку по формуле (11) для O или по формуле (12) для X .
7. Вернуть клетку с максимальной итоговой оценкой:

$$(i^*, j^*) = \arg \max_{(i,j) \in C} F_s(i, j).$$

2.9. Обоснование выбранного подхода

Полный перебор дерева игры на поле 20×20 приводит к слишком большому числу продолжений, поэтому в работе используется эвристический алгоритм. Он оценивает не всю будущую партию, а признаки, которые непосредственно влияют на ближайшие ходы: готовые линии, открытые отрезки, немедленные угрозы, двойные угрозы, положение относительно активной зоны и влияние препятствий. Такой подход не гарантирует оптимальную игру во всех позициях, но позволяет получать ход за малое время и сохранять разумное качество решений в длинных сериях партий.

3. Особенности реализации

Реализация выполнена на языке C++ в классе `MyPlayer`. Класс содержит два поля: `m_sign`, в котором хранится знак текущего игрока, и `m_name`, в котором хранится имя экземпляра игрока. Метод `set_sign` получает знак, назначенный игровым движком, метод `get_name` возвращает имя игрока, а метод `make_move` принимает состояние партии и возвращает координаты следующего хода. Интерфейс класса приведён в приложении А, а полный код реализации приведён в приложении В.

Внутри алгоритма состояние поля преобразуется во вспомогательную структуру `Board`. Она хранит число столбцов, число строк и последовательность значений клеток. Математически эта структура соответствует матрице A из формулы (1); одномерное хранение используется только как способ размещения данных в памяти. Доступ к клетке осуществляется по её координатам, а проверки допустимости хода сводятся к проверке принадлежности координат границам поля и проверке равенства клетки состоянию $\emptyset$.

Вспомогательные функции разделены по смыслу. Функции `inside`, `is_free` и `is_player_sign` проверяют границы поля, свободу клетки и принадлежность значения одному из игроков. Функция `get_candidate_cells` строит множество C из формулы (7). Функции `has_line_from_cell` и `has_line_after_move` проверяют наличие победной линии в четырёх направлениях из формулы (3). Функции `count_immediate_wins` и `count_immediate_wins_limited` вычисляют количество немедленных угроз $R_s(A)$, используемое в формулах (5) и (10).

Поиск крупнейшего свободного кластера реализован обходом в ширину. Для каждой найденной компоненты свободных клеток вычисляется размер и средняя точка, после чего выбирается свободная клетка, ближайшая к этой точке. При равенстве расстояний учитывается число возможных отрезков длины 5 и штраф за препятствия. Этот порядок соответствует математическому описанию из подраздела 2.3 и не зависит от особенностей C++.

Основной метод `make_move` реализует порядок, описанный в подразделе 2.8. Сначала он обрабатывает первый ход, затем ищет собственную немедленную победу, затем блокирует немедленную победу соперника, после чего оценивает оставшиеся кандидаты. Для игрока X отдельно сохраняется ход, который строит линию X , но позволяет O последним ходом добиться ничьей; такой ход используется как запасной вариант, если обычная оценка позиции

оказывается неблагоприятной.

Реализация является детерминированной: случайный выбор не используется, поэтому при одинаковом состоянии поля бот возвращает один и тот же ход. Это упрощает отладку и делает результаты серийных экспериментов воспроизводимыми.

4. Результаты работы программы

Для проверки бота были проведены автоматические серии игр в стандартном игровом движке проекта. Во всех сериях использовалось поле 20×20 с теми же правилами победы и препятствий, для которых разрабатывался алгоритм. Для каждого внешнего соперника запускались две серии по 1000 партий: в первой серии MyPlayer играл за X , во второй — за O . Дополнительно была проведена серия из 1000 партий MyPlayer против MyPlayer. Такая постановка эксперимента позволяет отдельно оценить влияние знака игрока, поскольку правила для X и O несимметричны.

В таблице 1 приведены абсолютные и процентные результаты игр. Проценты рассчитаны относительно 1000 партий в каждой серии.

Таблица 1: Результаты серийного тестирования бота MyPlayer

Серия	Знак	Победы MyPlayer	Победы соперника	Ничьи	Победы без ничьих
MyPlayer — BaselineEasy	X	819 / 81,9%	92 / 9,2%	89 / 8,9%	89,9%
BaselineEasy — MyPlayer	O	556 / 55,6%	334 / 33,4%	110 / 11,0%	62,5%
MyPlayer — BaselineHard	X	185 / 18,5%	232 / 23,2%	583 / 58,3%	44,4%
BaselineHard — MyPlayer	O	156 / 15,6%	502 / 50,2%	342 / 34,2%	23,7%
MyPlayer — MyPlayer	X/O	877 / 87,7%	119 / 11,9%	4 / 0,4%	88,1%

Результаты из таблицы 1 показывают, что против BaselineEasy разработанный бот имеет устойчивое преимущество. В сумме за две серии против этого соперника MyPlayer выиграл 1375 партий из 2000, то есть 68,75% всех партий. Если исключить ничьи, то на 1801 результативную партию приходится 1375 побед MyPlayer, что составляет 76,3% и превышает требуемые 50%. Против BaselineHard результат заметно слабее: всего было получено 341 победа MyPlayer, 734 победы соперника и 925 ничьих. Без учёта ничьих доля побед MyPlayer равна 31,7%, поэтому формальное требование 25% также выполнено, хотя при игре за O результат хуже, чем при игре за X . Это говорит о том, что локальные эвристики хорошо справляются с простым соперником, но против более сильной стратегии им не всегда хватает поиска на несколько ходов вперёд.

Необычный результат наблюдается в серии MyPlayer против MyPlayer: первый экземпляр бота выиграл 87,7% партий. Вероятное объяснение состоит в преимуществе первого хода и в агрессивном развитии инициативы после выбора активной области. Поскольку оба экземпляра используют одина-

ковую детерминированную стратегию, первый игрок раньше занимает центральные перспективные клетки и чаще первым создаёт угрозы, которые второму игроку приходится блокировать.

Время выбора хода измерялось средствами игрового движка. Работа была выполнена на ноутбуке Asus с процессором 13th Gen Intel(R) Core(TM) i7-1355U 1.70 GHz, оперативной памятью 32,00 ГБ и 64-разрядной операционной системой. Средние значения приведены в таблице 2.

Таблица 2: Среднее время расчёта хода и обработки партии

Серия	Ход MyPlayer, мс	Ход соперника, мс	Игровой процесс, мс
MyPlayer — BaselineEasy	3,4036	0,0025	1,7132
BaselineEasy — MyPlayer	3,3977	0,0025	1,7097
MyPlayer — BaselineHard	4,5028	0,0028	2,2596
BaselineHard — MyPlayer	4,5816	0,0029	2,3002
MyPlayer — MyPlayer	1,0336 / 1,1970	—	1,1185

Данные из таблицы 2 показывают, что даже при проверке немедленных угроз, обходе кандидатов и вычислении оценок отрезков длины 5 среднее время выбора хода остаётся в пределах нескольких миллисекунд. Это значительно меньше требуемого диапазона 50 — 100 мс на рабочей станции или ноутбуке. Время против BaselineEasy и BaselineHard оказалось больше, чем в режиме MyPlayer против MyPlayer, потому что партии против базовых игроков чаще уходили в глубокую стадию, когда почти всё поле уже занято и приходится анализировать больше сложных локальных конфигураций. В самоигре наблюдалась полная доминация первого игрока X , из-за чего партии завершались быстрее, а среднее число оцениваемых поздних позиций было меньше.

Заключение

В ходе работы был разработан бот MyPlayer для расширенной версии игры “Крестики-нолики” на поле 20×20 . Основным результатом стала детерминированная эвристическая стратегия, которая принимает на вход состояние поля и знак текущего игрока, строит множество ходов-кандидатов, проверяет немедленную победу и необходимость блокировки, а затем выбирает ход по оценочной функции. В алгоритме явно учтены препятствия, центр крупнейшего свободного кластера, потенциальные отрезки длины 5, двойные угрозы и асимметричное правило последнего хода для игрока X.

Практическая проверка показала, что бот уверенно превосходит простого соперника BaselineEasy: суммарно он выиграл 1375 партий из 2000, что составляет 68,75% всех партий и 76,3% результативных партий без учёта ничьих. Против BaselineHard суммарная доля побед без учёта ничьих составила 31,7%, поэтому формальное требование к игроку повышенного уровня сложности также выполнено. Среднее время выбора хода составило примерно 3,4 мс против BaselineEasy и 4,5—4,6 мс против BaselineHard, что значительно меньше требуемого времени порядка 50 — 100 мс на ноутбуке.

При выполнении работы были закреплены навыки формализации игровой задачи, проектирования эвристической функции, работы с матричным представлением поля, реализации обхода в ширину для поиска связных областей, обработки специальных правил игры и анализа результатов статистического тестирования. Главным ограничением решения остаётся отсутствие полноценного поиска в глубину: бот хорошо оценивает ближайшие угрозы, но может не видеть сложные многоходовые комбинации. Дальнейшее развитие алгоритма целесообразно связать с $\min\max$ -поиском по ограниченному множеству лучших кандидатов, автоматическим подбором весов и более точным различением открытых и закрытых линий.

Список литературы

1. Кормен Т. Х., Лейзерсон Ч. Е., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 3-е изд. М.: Вильямс, 2013. 1328 с.
2. Рассел С., Норвиг П. Искусственный интеллект: современный подход. 4-е изд. М.: Диалектика, 2021. 1152 с.
3. Beck J. Combinatorial Games: Tic-Tac-Toe Theory. Cambridge: Cambridge University Press, 2008. 732 p.
4. C++ reference: справочник по языку C++ [Электронный ресурс]. URL: <https://en.cppreference.com/w/> (дата обращения: 10.05.2026).

А. Исходный код заголовочного файла `my_player.hpp`

В приложении приведён полный заголовочный файл класса `MyPlayer`. Этот файл содержит объявление полей класса, конструктора и методов, которые используются игровым движком. Основная роль интерфейса описана в разделе 3, а сам интерфейс приведён в листинге 1.

```
1 #pragma once
2 #include "core/game.hpp"
3 namespace ttt::my_player {
4     using game::Event;
5     using game::IPlayer;
6     using game::Point;
7     using game::Sign;
8     using game::State;
9     class MyPlayer : public IPlayer {
10     Sign m_sign = Sign::NONE;
11     const char *m_name;
12     public:
13     MyPlayer(const char *name) : m_sign(Sign::NONE), m_name(name)
14     {}
15     void set_sign(Sign sign) override;
16     Point make_move(const State &game) override;
17     const char *get_name() const override;
18 };
19 }; // namespace ttt::my_player
```

Листинг 1: Файл `my_player.hpp`

В. Исходный код файла реализации my_player.cpp

В приложении приведён полный файл реализации разработанного бота. В нём находятся функции подготовки поля, поиска кандидатов, проверки линий, оценки позиции и выбора итогового хода. Подробное описание основных групп функций дано в разделе 3, а полный код реализации помещён в приложении В.

```
1 #include "my_player.hpp"
2 #include <cmath>
3 #include <limits>
4 #include <queue>
5 #include <vector>
6 namespace ttt::my_player {
7 static const int WIN_LENGTH = 5;
8 void MyPlayer::set_sign(Sign sign) { m_sign = sign; }
9 const char *MyPlayer::get_name() const { return m_name; }
10 struct Board {
11 int cols;
12 int rows;
13 std::vector<Sign> cells;
14 Sign get(int x, int y) const {
15 return cells[y * cols + x];
16 }
17 void set(int x, int y, Sign sign) {
18 cells[y * cols + x] = sign;
19 }
20 };
21 struct ClusterInfo {
22 bool valid;
23 int center_x;
24 int center_y;
25 int size;
26 };
27 static Point make_point(int x, int y);
28 static bool inside(const Board &board, int x, int y);
29 static bool is_free(const Board &board, int x, int y);
30 static bool is_player_sign(Sign sign);
31 static int count_possible_lines_through_cell(
32 const Board &board,
33 int x,
34 int y
35 );
36 static int obstacle_penalty_near_cell(
37 const Board &board,
38 int x,
39 int y
```

```

40 );
41 static ClusterInfo find_largest_free_cluster_center(const Board
42 &board);
43 static Sign get_enemy(Sign sign) {
44 if (sign == Sign::X)
45 return Sign::O;
46 if (sign == Sign::O)
47 return Sign::X;
48 return Sign::NONE;
49 }
50 static bool is_player_sign(Sign sign) {
51 return sign == Sign::X || sign == Sign::O;
52 }
53 static bool inside(const Board &board, int x, int y) {
54 return x >= 0 && x < board.cols && y >= 0 && y < board.rows;
55 }
56 static bool is_free(const Board &board, int x, int y) {
57 return inside(board, x, y) && board.get(x, y) == Sign::NONE;
58 }
59 static Point make_point(int x, int y) {
60 Point p;
61 p.x = x;
62 p.y = y;
63 return p;
64 }
65 static Board make_board(const State &state) {
66 Board board;
67 board.cols = state.get_opts().cols;
68 board.rows = state.get_opts().rows;
69 board.cells.resize(board.cols * board.rows);
70 for (int y = 0; y < board.rows; ++y) {
71 for (int x = 0; x < board.cols; ++x) {
72 board.set(x, y, state.get_value(x, y));
73 }
74 }
75 return board;
76 }
77 static bool board_empty(const Board &board) {
78 for (int y = 0; y < board.rows; ++y) {
79 for (int x = 0; x < board.cols; ++x) {
80 if (is_player_sign(board.get(x, y))) {
81 return false;
82 }
83 }
84 }
85 return true;
86 }
87 static int count_free_cells(const Board &board) {

```

```

88 int result = 0;
89 for (int y = 0; y < board.rows; ++y) {
90     for (int x = 0; x < board.cols; ++x) {
91         if (board.get(x, y) == Sign::NONE) {
92             ++result;
93         }
94     }
95 }
96 return result;
97 }
98 static bool has_neighbor(const Board &board, int x, int y, int
99 radius) {
100     for (int dy = -radius; dy <= radius; ++dy) {
101         for (int dx = -radius; dx <= radius; ++dx) {
102             if (dx == 0 && dy == 0)
103                 continue;
104             int nx = x + dx;
105             int ny = y + dy;
106             if (!inside(board, nx, ny))
107                 continue;
108             if (is_player_sign(board.get(nx, ny))) {
109                 return true;
110             }
111         }
112     }
113     return false;
114 }
115 static std::vector<Point> get_candidate_cells(const Board &board
116 ) {
117     std::vector<Point> result;
118     bool empty = board_empty(board);
119     for (int y = 0; y < board.rows; ++y) {
120         for (int x = 0; x < board.cols; ++x) {
121             if (!is_free(board, x, y))
122                 continue;
123             if (empty || has_neighbor(board, x, y, 2)) {
124                 result.push_back(make_point(x, y));
125             }
126         }
127     }
128     if (!result.empty()) {
129         return result;
130     }
131     for (int y = 0; y < board.rows; ++y) {
132         for (int x = 0; x < board.cols; ++x) {
133             if (is_free(board, x, y)) {
134                 result.push_back(make_point(x, y));
135             }

```

```

136 }
137 }
138 return result;
139 }
140 static bool has_line_from_cell(
141 const Board &board,
142 int x,
143 int y,
144 Sign sign
145 ) {
146     const int directions[4][2] = {
147     {1, 0},
148     {0, 1},
149     {1, 1},
150     {1, -1}
151 };
152     for (int d = 0; d < 4; ++d) {
153         int dx = directions[d][0];
154         int dy = directions[d][1];
155         int count = 1;
156         int nx = x + dx;
157         int ny = y + dy;
158         while (inside(board, nx, ny) && board.get(nx, ny) == sign) {
159             ++count;
160             nx += dx;
161             ny += dy;
162         }
163         nx = x - dx;
164         ny = y - dy;
165         while (inside(board, nx, ny) && board.get(nx, ny) == sign) {
166             ++count;
167             nx -= dx;
168             ny -= dy;
169         }
170         if (count >= WIN_LENGTH) {
171             return true;
172         }
173     }
174     return false;
175 }
176 static bool has_line_after_move(
177 const Board &board,
178 Point move,
179 Sign sign
180 ) {
181     Board copy = board;
182     copy.set(move.x, move.y, sign);
183     return has_line_from_cell(copy, move.x, move.y, sign);

```

```

184 }
185 static int count_immediate_wins(
186 const Board &board,
187 Sign sign
188 ) {
189 std::vector<Point> cells = get_candidate_cells(board);
190 int result = 0;
191 for (Point move : cells) {
192 if (has_line_after_move(board, move, sign)) {
193 ++result;
194 }
195 }
196 return result;
197 }
198 static int count_immediate_wins_limited(
199 const Board &board,
200 Sign sign,
201 int limit
202 ) {
203 std::vector<Point> cells = get_candidate_cells(board);
204 int result = 0;
205 for (Point move : cells) {
206 if (has_line_after_move(board, move, sign)) {
207 ++result;
208 if (result >= limit) {
209 return result;
210 }
211 }
212 }
213 return result;
214 }
215 // X поставил линию.
216 // Это реальная победа X только если O не может последним ходом собрать
217 линию.
218 static bool is_real_x_win_after_move(
219 const Board &board,
220 Point move
221 ) {
222 Board after_x = board;
223 after_x.set(move.x, move.y, Sign::X);
224 if (!has_line_from_cell(after_x, move.x, move.y, Sign::X)) {
225 return false;
226 }
227 // Если поле заполнено, O не может сделать последний ход.
228 if (count_free_cells(after_x) == 0) {
229 return true;
230 }
231 int o_answers = count_immediate_wins_limited(after_x, Sign::O,

```

```

232 1);
233 return o_answers == 0;
234 }
235 // X поставил линию, но O может ответить своей линией.
236 // Это не победа X, а гарантированная ничья.
237 static bool is_x_draw_after_move(
238     const Board &board,
239     Point move
240 ) {
241     Board after_x = board;
242     after_x.set(move.x, move.y, Sign::X);
243     if (!has_line_from_cell(after_x, move.x, move.y, Sign::X)) {
244         return false;
245     }
246     if (count_free_cells(after_x) == 0) {
247         return false;
248     }
249     int o_answers = count_immediate_wins_limited(after_x, Sign::O,
250     1);
251     return o_answers > 0;
252 }
253 static long long pattern_score(
254     int own_count,
255     int empty_count,
256     int blocked_count
257 ) {
258     if (blocked_count > 0) {
259         return 0;
260     }
261     if (own_count >= 5) {
262         return 100000000LL;
263     }
264     if (own_count == 4 && empty_count == 1) {
265         return 3000000LL;
266     }
267     if (own_count == 3 && empty_count == 2) {
268         return 80000LL;
269     }
270     if (own_count == 2 && empty_count == 3) {
271         return 3000LL;
272     }
273     if (own_count == 1 && empty_count == 4) {
274         return 30LL;
275     }
276     return 0;
277 }
278 static long long evaluate_board_for_sign(
279     const Board &board,

```



```

280 Sign sign
281 ) {
282     const int directions[4][2] = {
283         {1, 0},
284         {0, 1},
285         {1, 1},
286         {1, -1}
287     };
288     Sign enemy = get_enemy(sign);
289     long long score = 0;
290     for (int y = 0; y < board.rows; ++y) {
291         for (int x = 0; x < board.cols; ++x) {
292             for (int d = 0; d < 4; ++d) {
293                 int dx = directions[d][0];
294                 int dy = directions[d][1];
295                 int end_x = x + (WIN_LENGTH - 1) * dx;
296                 int end_y = y + (WIN_LENGTH - 1) * dy;
297                 if (!inside(board, end_x, end_y))
298                     continue;
299                 int own_count = 0;
300                 int empty_count = 0;
301                 int blocked_count = 0;
302                 for (int i = 0; i < WIN_LENGTH; ++i) {
303                     int nx = x + i * dx;
304                     int ny = y + i * dy;
305                     Sign cell = board.get(nx, ny);
306                     if (cell == sign) {
307                         ++own_count;
308                     } else if (cell == Sign::NONE) {
309                         ++empty_count;
310                     } else if (cell == enemy) {
311                         ++blocked_count;
312                     } else {
313                         ++blocked_count;
314                     }
315                 }
316                 score += pattern_score(own_count, empty_count,
317                     blocked_count);
318             }
319         }
320     }
321     return score;
322 }
323 static int neighbor_bonus(
324     const Board &board,
325     Point move,
326     Sign my_sign
327 ) {

```

```

328 int bonus = 0;
329 Sign enemy = get_enemy(my_sign);
330 for (int dy = -2; dy <= 2; ++dy) {
331     for (int dx = -2; dx <= 2; ++dx) {
332         if (dx == 0 && dy == 0)
333             continue;
334         int x = move.x + dx;
335         int y = move.y + dy;
336         if (!inside(board, x, y))
337             continue;
338         Sign cell = board.get(x, y);
339         int distance = std::abs(dx) + std::abs(dy);
340         int value = distance <= 1 ? 30 : 8;
341         if (cell == my_sign) {
342             bonus += value;
343         } else if (cell == enemy) {
344             bonus += value / 2;
345         }
346     }
347 }
348 return bonus;
349 }
350 static int strategic_center_bonus(
351     Point move,
352     const ClusterInfo &cluster
353 ) {
354     if (!cluster.valid) {
355         return 0;
356     }
357     int distance =
358         std::abs(move.x - cluster.center_x) +
359         std::abs(move.y - cluster.center_y);
360     return -distance;
361 }
362 // Специальный выбор атакующего хода за O.
363 // Ищем ход, после которого у O появляется одна или несколько
364 // немедленных победных угроз на следующий ход.
365 // При этом не выбираем ход, после которого X сразу получает линию
366 .
367 static bool find_o_attack_move(
368     const Board &board,
369     const std::vector<Point> &cells,
370     const ClusterInfo &cluster,
371     Point &best_move
372 ) {
373     bool found = false;
374     long long best_score = std::numeric_limits<long long>::min();
375     for (Point move : cells) {

```

```

376 Board after_o = board;
377 after_o.set(move.x, move.y, Sign::O);
378 // Если после нашего хода X сразу может собрать линию,
379 // такой ход слишком опасен.
380 int x_wins = count_immediate_wins_limited(after_o, Sign::X,
381 1);
382 if (x_wins > 0) {
383     continue;
384 }
385 int o_wins = count_immediate_wins_limited(after_o, Sign::O,
386 2);
387 if (o_wins == 0) {
388     continue;
389 }
390 long long score = 0;
391 // Две и более угрозы почти всегда сильнее одной.
392 if (o_wins >= 2) {
393     score += 150000000LL;
394 } else {
395     score += 30000000LL;
396 }
397 score += evaluate_board_for_sign(after_o, Sign::O) * 12;
398 score -= evaluate_board_for_sign(after_o, Sign::X) * 8;
399 score += neighbor_bonus(board, move, Sign::O);
400 score += strategic_center_bonus(move, cluster);
401 if (!found || score > best_score) {
402     found = true;
403     best_score = score;
404     best_move = move;
405 }
406 }
407 return found;
408 }
409 static long long evaluate_move(
410     const Board &board,
411     Point move,
412     Sign my_sign,
413     const ClusterInfo &cluster
414 ) {
415     Sign enemy = get_enemy(my_sign);
416     Board after_my_move = board;
417     after_my_move.set(move.x, move.y, my_sign);
418     // Особое правило:
419     // O выигрывает сразу, X выигрывает только если O не может ответить
420     // линией.
421     if (my_sign == Sign::O) {
422         if (has_line_from_cell(after_my_move, move.x, move.y, Sign::
423         O)) {

```

```

424 return 1000000000LL;
425 }
426 }
427 if (my_sign == Sign::X) {
428 if (has_line_from_cell(after_my_move, move.x, move.y, Sign::
429 X)) {
430 if (count_free_cells(after_my_move) == 0) {
431 return 1000000000LL;
432 }
433 int o_answers = count_immediate_wins_limited(after_my_move
434 , Sign::O, 1);
435 if (o_answers == 0) {
436 return 1000000000LL;
437 }
438 // Это не победа, а ничья. Хорошо как fallback,
439 // но не считаем это абсолютной победой.
440 return 10000000LL;
441 }
442 }
443 long long score = 0;
444 int enemy_immediate_wins = count_immediate_wins(after_my_move,
445 enemy);
446 int my_immediate_wins_next = count_immediate_wins(
447 after_my_move, my_sign);
448 if (enemy_immediate_wins > 0) {
449 if (my_sign == Sign::O) {
450 score -= 120000000LL * enemy_immediate_wins;
451 } else {
452 score -= 50000000LL * enemy_immediate_wins;
453 }
454 }
455 if (my_sign == Sign::O) {
456 if (my_immediate_wins_next >= 2) {
457 score += 150000000LL;
458 } else if (my_immediate_wins_next == 1) {
459 score += 30000000LL;
460 }
461 } else {
462 if (my_immediate_wins_next >= 2) {
463 score += 30000000LL;
464 } else if (my_immediate_wins_next == 1) {
465 score += 3000000LL;
466 }
467 }
468 long long attack = evaluate_board_for_sign(after_my_move,
469 my_sign);
470 long long defense = evaluate_board_for_sign(after_my_move,
471 enemy);

```

```

472 if (my_sign == Sign::O) {
473 // O должен атаковать, но не может игнорировать угрозы X.
474 // При слабой защите O слишком часто проигрывает.
475 score += attack * 22;
476 score -= defense * 24;
477 } else {
478 // X должен защищаться сильнее, потому что после линии X у O есть
479 последний ход.
480 score += attack * 12;
481 score -= defense * 24;
482 }
483 score += neighbor_bonus(board, move, my_sign);
484 score += strategic_center_bonus(move, cluster);
485 return score;
486 }
487 static int count_possible_lines_through_cell(
488 const Board &board,
489 int x,
490 int y
491 ) {
492 const int directions[4][2] = {
493 {1, 0},
494 {0, 1},
495 {1, 1},
496 {1, -1}
497 };
498 int result = 0;
499 for (int d = 0; d < 4; ++d) {
500 int dx = directions[d][0];
501 int dy = directions[d][1];
502 for (int shift = -(WIN_LENGTH - 1); shift <= 0; ++shift) {
503 int start_x = x + shift * dx;
504 int start_y = y + shift * dy;
505 bool valid = true;
506 for (int i = 0; i < WIN_LENGTH; ++i) {
507 int nx = start_x + i * dx;
508 int ny = start_y + i * dy;
509 if (!inside(board, nx, ny) || board.get(nx, ny) != Sign
510 ::NONE) {
511 valid = false;
512 break;
513 }
514 }
515 if (valid) {
516 ++result;
517 }
518 }
519 }

```

```

520 return result;
521 }
522 static int obstacle_penalty_near_cell(
523     const Board &board,
524     int x,
525     int y
526 ) {
527     int penalty = 0;
528     for (int dy = -2; dy <= 2; ++dy) {
529         for (int dx = -2; dx <= 2; ++dx) {
530             if (dx == 0 && dy == 0)
531                 continue;
532             int nx = x + dx;
533             int ny = y + dy;
534             if (!inside(board, nx, ny))
535                 continue;
536             Sign cell = board.get(nx, ny);
537             if (cell != Sign::NONE && cell != Sign::X && cell != Sign
538                 ::O) {
539                 int distance = std::abs(dx) + std::abs(dy);
540                 if (distance <= 1) {
541                     penalty += 20;
542                 } else {
543                     penalty += 7;
544                 }
545             }
546         }
547     }
548     return penalty;
549 }
550 static ClusterInfo find_largest_free_cluster_center(const Board
551 &board) {
552     ClusterInfo best_cluster;
553     best_cluster.valid = false;
554     best_cluster.center_x = 0;
555     best_cluster.center_y = 0;
556     best_cluster.size = 0;
557     std::vector<int> visited(board.cols * board.rows, 0);
558     const int directions[8][2] = {
559         {1, 0},
560         {-1, 0},
561         {0, 1},
562         {0, -1},
563         {1, 1},
564         {1, -1},
565         {-1, 1},
566         {-1, -1}
567     };

```

```

568 for (int y = 0; y < board.rows; ++y) {
569 for (int x = 0; x < board.cols; ++x) {
570 if (!is_free(board, x, y)) {
571 continue;
572 }
573 int start_index = y * board.cols + x;
574 if (visited[start_index]) {
575 continue;
576 }
577 std::queue<Point> queue;
578 std::vector<Point> component;
579 visited[start_index] = 1;
580 queue.push(make_point(x, y));
581 long long sum_x = 0;
582 long long sum_y = 0;
583 while (!queue.empty()) {
584 Point current = queue.front();
585 queue.pop();
586 component.push_back(current);
587 sum_x += current.x;
588 sum_y += current.y;
589 for (int d = 0; d < 8; ++d) {
590 int nx = current.x + directions[d][0];
591 int ny = current.y + directions[d][1];
592 if (!inside(board, nx, ny)) {
593 continue;
594 }
595 int index = ny * board.cols + nx;
596 if (visited[index]) {
597 continue;
598 }
599 if (!is_free(board, nx, ny)) {
600 continue;
601 }
602 visited[index] = 1;
603 queue.push(make_point(nx, ny));
604 }
605 }
606 int component_size = static_cast<int>(component.size());
607 if (component_size == 0) {
608 continue;
609 }
610 Point center_cell = component.front();
611 long long best_distance = std::numeric_limits<long long>::
612 max();
613 long long best_cell_score = std::numeric_limits<long long
614 >::min();
615 for (Point cell : component) {

```

```

616 long long dx = static_cast<long long>(cell.x) *
617 component_size - sum_x;
618 long long dy = static_cast<long long>(cell.y) *
619 component_size - sum_y;
620 long long distance = dx * dx + dy * dy;
621 int possible_lines = count_possible_lines_through_cell(
622 board,
623 cell.x,
624 cell.y
625 );
626 int obstacle_penalty = obstacle_penalty_near_cell(
627 board,
628 cell.x,
629 cell.y
630 );
631 long long cell_score = 0;
632 cell_score += possible_lines * 1000LL;
633 cell_score -= obstacle_penalty * 20LL;
634 if (distance < best_distance ||
635 (distance == best_distance && cell_score >
636 best_cell_score)) {
637 best_distance = distance;
638 best_cell_score = cell_score;
639 center_cell = cell;
640 }
641 }
642 if (!best_cluster.valid || component_size > best_cluster.
643 size) {
644 best_cluster.valid = true;
645 best_cluster.center_x = center_cell.x;
646 best_cluster.center_y = center_cell.y;
647 best_cluster.size = component_size;
648 }
649 }
650 }
651 return best_cluster;
652 }
653 static Point choose_first_move(
654 const Board &board,
655 const ClusterInfo &cluster
656 ) {
657 Point best = make_point(0, 0);
658 long long best_score = std::numeric_limits<long long>::min();
659 for (int y = 0; y < board.rows; ++y) {
660 for (int x = 0; x < board.cols; ++x) {
661 if (!is_free(board, x, y))
662 continue;
663 int possible_lines = count_possible_lines_through_cell(

```



```

664 board, x, y);
665 if (possible_lines == 0)
666     continue;
667 int obstacle_penalty = obstacle_penalty_near_cell(board, x
668 , y);
669 int distance_to_cluster_center = 0;
670 if (cluster.valid) {
671     distance_to_cluster_center =
672     std::abs(x - cluster.center_x) +
673     std::abs(y - cluster.center_y);
674 }
675 long long score = 0;
676 score += possible_lines * 1000LL;
677 score -= obstacle_penalty * 20LL;
678 score -= distance_to_cluster_center;
679 if (score > best_score) {
680     best_score = score;
681     best = make_point(x, y);
682 }
683 }
684 }
685 return best;
686 }
687 Point MyPlayer::make_move(const State &state) {
688     Board board = make_board(state);
689     ClusterInfo free_cluster = find_largest_free_cluster_center(
690     board);
691     std::vector<Point> cells = get_candidate_cells(board);
692     if (cells.empty()) {
693         if (free_cluster.valid) {
694             return make_point(free_cluster.center_x, free_cluster.
695             center_y);
696         }
697         return make_point(0, 0);
698     }
699     if (board_empty(board)) {
700         return choose_first_move(board, free_cluster);
701     }
702     Point draw_by_x = cells.front();
703     bool has_draw_by_x = false;
704     // 1. Немедленная победа с учетом особого правила X/O.
705     if (m_sign == Sign::O) {
706         // O выигрывает сразу.
707         for (Point move : cells) {
708             if (has_line_after_move(board, move, Sign::O)) {
709                 return move;
710             }
711         }

```

```

712 } else if (m_sign == Sign::X) {
713 // X выигрывает только если O не сможет ответить своейлинией
714 .
715 for (Point move : cells) {
716 if (is_real_x_win_after_move(board, move)) {
717 return move;
718 }
719 }
720 // Если есть ход X в линию, но O может ответить линией,
721 // это гарантированная ничья. Запоминаем как fallback.
722 for (Point move : cells) {
723 if (is_x_draw_after_move(board, move)) {
724 draw_by_x = move;
725 has_draw_by_x = true;
726 break;
727 }
728 }
729 }
730 // 2. Блок немедленной победы соперника.
731 // Для X это критично: O выигрывает сразу.
732 // Для O тоже блокируем линию X, потому что иначе X получаетсильнейший
733 темп,
734 // а O часто не успевает реализовать ответ.
735 if (m_sign == Sign::X) {
736 for (Point move : cells) {
737 if (has_line_after_move(board, move, Sign::O)) {
738 return move;
739 }
740 }
741 } else if (m_sign == Sign::O) {
742 // Сначала обязательно блокируем немедленную линию X.
743 for (Point move : cells) {
744 if (has_line_after_move(board, move, Sign::X)) {
745 return move;
746 }
747 }
748 // Если срочного блока нет, ищем активный атакующий ход за O
749 .
750 Point o_attack_move = cells.front();
751 if (find_o_attack_move(board, cells, free_cluster,
752 o_attack_move)) {
753 return o_attack_move;
754 }
755 }
756 Point best_move = cells.front();
757 long long best_score = std::numeric_limits<long long>::min();
758 for (Point move : cells) {
759 long long score = evaluate_move(board, move, m_sign,

```

```

760 free_cluster);
761 if (score > best_score) {
762     best_score = score;
763     best_move = move;
764 }
765 }
766 // Если мы играем за X и обычная эвристика видит плохую позицию
767 ,
768 // но есть ход, который гарантирует ничью, выбираем ничью.
769 if (m_sign == Sign::X && has_draw_by_x && best_score < 0) {
770     return draw_by_x;
771 }
772 return best_move;
773 }
774 } // namespace ttt::my_player
775 46__

```

Листинг 2: Файл my_player.cpp